```
!pip install transformers datasets torch scikit-learn pandas
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transfo
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from tran
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from tran
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from sci
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from sc
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pan
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from p
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingfac
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from req
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from ai
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from ht
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpco
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typ
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typ
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (
```

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArgu
from datasets import load_dataset
```

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import numpy as np
```

```
dataset = load_dataset("imdb")

print(dataset)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingfac
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datas
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable h
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
README.md: 7.81k/? [00:00<00:00, 188kB/s]
plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 105MB/s]
plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 102MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:00<00:00, 109MB/s]
Generating train split: 100% 25000/25000 [00:00<00:00, 83628.70 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 108054.28 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 124394.53 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
```

```
print(dataset['train'][0])
```

```
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that sur
```

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

```
config.json: 100% 483/483 [00:00<00:00, 45.5kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.63kB/s]
vocab.txt: 232k/? [00:00<00:00, 6.94MB/s]
tokenizer.json: 466k/? [00:00<00:00, 14.4MB/s]
```

```
def tokenize_function(example):
    return tokenizer(example["text"], padding="max_length", truncation=True)
```

```
tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```
Map: 100%                                    25000/25000 [00:30<00:00, 824.77 examples/s]
Map: 100%                                    25000/25000 [00:37<00:00, 808.85 examples/s]
Map: 100%                                    50000/50000 [01:00<00:00, 889.97 examples/s]
```

```
tokenized_datasets = tokenized_datasets.remove_columns(["text"])
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format("torch")
```

```
print(tokenized_datasets["train"][0])
```

```
{'labels': tensor(0), 'input_ids': tensor([  101,  1045, 12524,  1045,  2572,  8025,  1011,  375
         2678,  3573,  2138,  1997,  2035,  1996,  6704,  2008,  5129,  2009,
         2043,  2009,  2001,  2034,  2207,  1999,  3476,  1012,  1045,  2036,
         2657,  2008,  2012,  2034,  2009,  2001,  8243,  2011,  1057,  1012,
         1055,  1012,  8205,  2065,  2009,  2412,  2699,  2000,  4607,  2023,
         2406,  1010,  3568,  2108,  1037,  5470,  1997,  3152,  2641,  1000,
         6801,  1000,  1045,  2428,  2018,  2000,  2156,  2023,  2005,  2870,
         1012,  1026,  7987,  1013,  1028,  1026,  7987,  1013,  1028,  1996,
         5436,  2003,  8857,  2105,  1037,  2402,  4467,  3689,  3076,  2315,
        14229,  2040,  4122,  2000,  4553,  2673,  2016,  2064,  2055,  2166,
         1012,  1999,  3327,  2016,  4122,  2000,  3579,  2014,  3086,  2015,
         2000,  2437,  2070,  4066,  1997,  4516,  2006,  2054,  1996,  2779,
        25430, 14728,  2245,  2055,  3056,  2576,  3314,  2107,  2004,  1996,
         5148,  2162,  1998,  2679,  3314,  1999,  1996,  2142,  2163,  1012,
         1999,  2090,  4851,  8801,  1998,  6623,  7939,  4697,  3619,  1997,
         8947,  2055,  2037, 10740,  2006,  4331,  1010,  2016,  2038,  3348,
         2007,  2014,  3689,  3836,  1010, 19846,  1010,  1998,  2496,  2273,
         1012,  1026,  7987,  1013,  1028,  1026,  7987,  1013,  1028,  2054,
         8563,  2033,  2055,  1045,  2572,  8025,  1011,  3756,  2003,  2008,
         2871,  2086,  3283,  1010,  2023,  2001,  2641, 26932,  1012,  2428,
         1010,  1996,  3348,  1998, 16371, 25469,  5019,  2024,  2261,  1998,
         2521,  2090,  1010,  2130,  2059,  2009,  1005,  1055,  2025,  2915,
         2066,  2070, 10036,  2135,  2081, 22555,  2080,  1012,  2096,  2026,
         2406,  3549,  2568,  2424,  2009, 16880,  1010,  1999,  4507,  3348,
         1998, 16371, 25469,  2024,  1037,  2350, 18785,  1999,  4467,  5988,
         1012,  2130, 13749,  7849, 24544,  1010, 15835,  2037,  3437,  2000,
         2204,  2214,  2879,  2198,  4811,  1010,  2018,  3348,  5019,  1999,
         2010,  3152,  1012,  1026,  7987,  1013,  1028,  1026,  7987,  1013,
         1028,  1045,  2079,  4012,  3549,  2094,  1996, 16587,  2005,  1996,
         2755,  2008,  2151,  3348,  3491,  1999,  1996,  2143,  2003,  3491,
         2005,  6018,  5682,  2738,  2084,  2074,  2000,  5213,  2111,  1998,
         2191,  2769,  2000,  2022,  3491,  1999, 26932, 12370,  1999,  2637,
         1012,  1045,  2572,  8025,  1011,  3756,  2003,  1037,  2204,  2143,
         2005,  3087,  5782,  2000,  2817,  1996,  6240,  1998, 14629,  1006,
         2053, 26136,  3832,  1007,  1997,  4467,  5988,  1012,  2021,  2428,
         1010,  2023,  2143,  2987,  1005,  1056,  2031,  2172,  1997,  1037,
         5436,  1012,   102,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
```

```
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0]), 'token_type_ids': tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

```
small_train_dataset = tokenized_datasets["train"].shuffle(seed=42).select(range(2000))
small_test_dataset = tokenized_datasets["test"].shuffle(seed=42).select(range(1000))
```

```
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=
```

```
model.safetensors: 100%                                                    268M/268M [00:01<00:00, 264MB/s]

Loading weights: 100%   100/100 [00:00<00:00, 372.77it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.w

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expec
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider tra
```

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = np.argmax(pred.predictions, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')
    acc = accuracy_score(labels, preds)

    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall
    }
```

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
)
```

```
trainer = Trainer(
    model=model,
```

```
        args=training_args,
        train_dataset=small_train_dataset,
        eval_dataset=small_test_dataset,
        compute_metrics=compute_metrics
    )
```

```
text = "This movie was absolutely amazing!"

inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)

outputs = model(**inputs)

pred = torch.argmax(outputs.logits)

if pred == 1:
    print("Positive Sentiment")
else:
    print("Negative Sentiment")
```

```
Positive Sentiment
```

```
import pandas as pd

# Evaluate the model
results = trainer.evaluate()

# Create table
data = {
    "Metric": ["Accuracy", "Precision", "Recall", "F1 Score"],
    "Score": [
        results["eval_accuracy"],
        results["eval_precision"],
        results["eval_recall"],
        results["eval_f1"]
    ]
}

result_table = pd.DataFrame(data)

print(result_table)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memc
  super().__init__(loader)
```

[84/125 09:41 < 04:47, 0.14 it/s]